Progress Report

CSC 503, Winter 2025

Nitin Gupta, Akina Kuperus, Taimoor Shahzad, Riya Shrestha, Abdul-Wahid Yakubu

1. Problem

The main setback we have encountered on this project is with overfitting and poor generalisation across several models and feature vectorization techniques. Our group has implemented several preprocessing techniques on the EmpatheticDialouges dataset which slightly increased our accuracy at the initial stage on training data, but further refinement is needed. Several feature extraction methods, such as Word2Vec, Doc2Vec and TF-IDF, have been tested with Logistic Regression, Support Vector Machines (SVM), Naives Bayes, XGBoost etc. Whilst some configurations have produced somewhat minimal training error for the preprocessing we have done, validation error remains high, indicating that the models are unable to generalise well to unseen data.

One experiment varied the ngram_range hyperparameter between (1,1) to (1,3) in CountVectorizer, lowering the training error to 5% but leaving the validation error high highlighting issues with overfitting. This shows that the current preprocessing pipeline may require additional optimisation to improve feature representation. Furthermore, Random Forest with TF-IDF produced a very low misclassification error of 0.009 on training data whilst failing to improve validation error. These findings highlight the ongoing challenge of balancing model complexity and feature representation to improve generalization. From here on out, the team will look into different preprocessing techniques. Additionally we will experiment with different hyperparameter tuning or regularization approaches to increase validation performance.

2. Goals

The main goal of this project is to classify empathetic text using the “Empathetic Dialogues” dataset (Rashkin et al.). The evaluation of our classification will be two-fold: training, validation, and test errors, and confusion matrices. Errors will be measured using the 0/1 loss function where a correctly labelled utterance has 0 error and an incorrectly labelled utterance has an error of 1. A purely random assignment of labels over 32 emotion classes would give an error of 31/32 or .96. The initial measure of success for our classification model is if the 95% confidence interval for the error does not contain .96. Then we can conclude with 95% confidence that our model performs better than random. This is merely a baseline measure of success. Ideally we achieve errors much lower. We will also use confidence intervals to compare the success of each classification model.

Confusion matrices will highlight which emotions are difficult to classify. In particular, we can see if two emotions are commonly getting confused by the classification. If, for example, “excited” and “disappointed” are often being misclassified as each other, it is more unsuccessful than if “terrified” and “afraid” are being misclassified as each other.

Initially, our project aimed to both classify and rewrite empathetic text using the "Empathetic Dialogues" dataset. However, to reduce complexity and focus on a more manageable scope, we have decided to concentrate solely on classification. The text rewriting component, including BERT-based response generation and potential reinforcement learning fine-tuning, has been removed.

3. Plan and Progress-to-date

3.1 Preprocessing

The dataset used for this project is the *Empathetic Dialogues* dataset, which consists of conversations where one speaker shares an experience, and another responds empathetically. Each conversation is labeled with one of 32 different emotion categories, such as *sadness, excitement, gratitude*, and *fear*. The dataset contains multiple features, but we focus on two key ones:

1. **Utterance** – This represents a single turn in the conversation, meaning one message from a speaker.
2. **Context** – This includes the emotion label for this conversation.

For example, consider this conversation snippet:

- **Utterance 1:** *"I just got a promotion at work!"*
- **Utterance 2:** *"Wow, that's amazing! You must be so excited!"*
- **Utterance 3:** *"Yes, I worked really hard for this!"*

Here, each utterance contributes to the overall emotional tone of the conversation, which might be labeled as *excited*. Instead of classifying emotions at the individual utterance level, we aim to classify the overall emotion of the conversation by analyzing all utterances together.

To prepare the dataset for emotion classification, we combined all utterances within a

conversation into a single text block. This ensures that our classifier captures the full emotional context rather than analyzing isolated sentences. We then removed stopwords—common words like "the," "is," and "and"—to focus on meaningful content. After that, we applied stemming, which reduces words to their root forms (e.g., "running" → "run") to standardize different variations of the same word. Finally, we tokenized the text, breaking it into individual words for further processing. These preprocessing steps enhance the dataset's suitability for training an effective emotion classification model by ensuring that it captures the broader sentiment of entire conversations.

3.2 Feature vectorization

We have done initial testing using three feature vectorization methods:

3.2.1 TF-IDF:

TF-IDF stands for Term Frequency-Inverse Document Frequency. TF-IDF Vectorization is a statistical measure used to evaluate the importance of a word in a document relative to a collection of documents (corpus). It is frequently used in text mining and information retrieval. Unlike simple word frequency, TF-IDF balances common and rare words to highlight the most meaningful terms.

1) Term Frequency (TF):

- Measures how frequently a term appears in a document. ● A higher TF indicates higher importance for the term.

$$\mathrm{TF}(t,d) = \frac{\text{Number of times term } t \text{ appears in document } d}{\text{Total number of terms in document } d}$$

2) Inverse Document Frequency (IDF):

- Measures the importance of a term across the entire corpus.
- A higher IDF indicates that the term is rare in the document and thus more significant.

$$\mathrm{IDF}(t,D) = \log\left(\frac{\text{Total number of documents in corpus } D}{\text{Number of documents containing term } t}\right)$$

3) TF-IDF:

- Words with high TF-IDF scores are considered important in a document but not overly common across the corpus.

$$\text{TF-IDF}(t,d,D) = \mathrm{TF}(t,d) \times \mathrm{IDF}(t,D)$$

In our case, we used TfidfVectorizer of Scikit-learn for converting the empathetic conversations into a matrix of TF-IDF features. This built-in vectorizer has multiple parameters that can be tuned to achieve a greater degree of pre-processing for the dataset. So far, we have tried adjusting following parameters:

- max_features - Limits the number of features (terms) to the top max_features based on term frequency.
  - For reference, there are 17844 conversations in the training dataset and each conversation has nearly 1 million features after "TfidfVectorizer" and without limiting max_features.
- Using max_features = 10000, 20000, 30000 is producing better results.
- ngram_range - Specifies the range of n-grams to generate i.e. unigrams, bigrams, trigrams. We have experimented with different parameters and found that preserving unigrams and trigrams (1,3) produces better results.
- stop_words - Specifies stop words to be removed. So far, we have used a built-in list of "English" stop words.

3.2.2 Word2Vec:

Word2Vec is a word embedding method that allows words to be represented as vectors. In particular, Word2Vec maps words with similar meanings to similar vectors using neural networks. This model functions under the assumption that words with similar meanings occur in similar contexts and employs two neural network algorithms: Continuous bag of words (CBOW) and skip-grams (Rong). These two architectures work as inverses. CBOW uses the surrounding context words to predict the current word. Skip-grams use the current word to predict the context words. Both of these architectures are used in the *gensim* (Řehůřek) implementation of Word2Vec.

For CBOW, given a vocabulary $V = \{w_1, w_2, \ldots, w_V\}$, the neural network input is a one-hot encoded vector of the context word . For simplicity in explanation, we assume for each word there is one context word. In this case, the input vector is $x = \{x_1, \ldots, x_V\}$ where $x_c = 1$ for all i except corresponding to w_c. There is one hidden layer with N nodes and an output layer $y_1, \ldots, y_n$, corresponding to the V words in the vocabulary. The output of CBOW is the probability vector where each element of the vector corresponds to the probability of a word given w_c is its context word.

For the skip-gram model we are now predicting the context word given the target word. If there is only one context word, the construction of the neural network is the same as for CBOW. The case with more than one context word will be described in the final report.

3.2.3 Doc2Vec Method:

The Doc2Vec method embeds a document or paragraph as a vector(Le et. al.). In *gensim* this is implemented with two models, analogous to the Word2Vec method. The first is the Paragraph Vector - Distributed Memory (PV-DM) which is an extension of the CBOW. The PV-DM neural network takes as an input the concatenation of the paragraph vector and the context word vectors and outputs a paragraph vector that incorporates the context. The training of this neural network is analogous to the Word2Vec case.

The Doc2Vec method also uses the Paragraph Vector - Distributed Bag of Words (PV-DBOW) algorithm which is similar to the skip-grams case. Here the neural network is given a paragraph vector and predicts words that would be randomly sampled from that paragraph.

After training, these paragraph vectors are used as inputs to linear SVM and logistic regression machine learning models.

3.3 Testing different methods:

We have done initial testing on several different methods:

A. Logistic regression:
The logistic regression model was implemented with L2 regularization. The regularization constant C was tuned using 5-fold cross-validation. The feature vectors were modelled using Word2Vec and Doc2Vec.

B. SVM-Linear:
A linear support vector machine (SVM) was implemented using the SGDClassifier with hinge loss and L2 regularization in *sklearn*. Again, the regularization constant alpha was tuned using 5-fold cross-validation. The feature vectors were modelled using Word2Vec and Doc2Vec.

C. Random Forest:
Random Forest Classifier was implemented to classify the emotions in the dataset. The text data was converted into numerical representations using TF-IDF Vectorization. The dataset is then split into training (80%) and validation (20%) sets to evaluate the model performance. The results show a training error of 0.3868 and a validation error of 0.6609. This indicates that the model effectively learns from the training data but badly in validation sets. This could be most likely due to overfitting.

D. Naive Bayes:
The model shows a final training error of 0.1125, indicating that the model performed well on the training data, correctly classifying approx. 88.75% of the samples. However, the validation error is

quite high (0.5802) which means the model struggles to generalize the unseen data with a 58.02% error rate on the validation set. This suggests that the model is overfitting to the training data, likely capturing noise that does not translate well to new unseen examples.

E. Logistic Regression - Balanced Class Weight:

In this model, the preprocessing was performed using TF-IDF Vectorizer with max_features=10,000 and n_grams = (1,3). For logistic regression, the "saga" solver was used as it is preferred for efficiently handling large datasets and faster convergence. The regularization parameter was set to C=10. Since our dataset contains an imbalance of classes for certain emotions, we also set class_weight = "balanced" to improve model performance. Without balancing, the model might ignore the minority class, leading to poor performance on it. Balancing ensures that the model learns to classify both majority and minority classes effectively without the need of resampling techniques like SMOTE. The training error with these parameters was "0.003" indicating high performance on the training set. The validation error was found to be "0.4683" which indicates that the model overfits on the training data but performs poorly on unseen examples.

F. Random Forest - Balanced Class Weight:

Preprocessing was performed using TF-IDF Vectorizer with max_features=50,000 and n_grams = (1,3). The forest size was set to n_estimators = 300, the number of features for splitting at a node max_features="sqrt", and the maximum tree depth was set to max_depth = 50. The class_weight = "balanced" was used to improve model performance. The training error was "0.0074" indicating high performance on the training set. The validation error was found to be "0.5458" which indicates that the model overfits on the training data but performs poorly on unseen examples.

G. SVM-RBF Kernel:

Preprocessing was performed using TF-IDF Vectorizer with max_features=10,000 and n_grams = (1,3). To deal with class imbalance, the SMOTE method with sampling_strategy = "not minority" was used to oversample the minority classes to match the majority classes. For the classifier, the regularization parameter was set to C=1, bandwidth set to gamma=1, and kernel= "rbf". With these parameters, the training error was "0.0102" and the validation error was "0.4987", indicating the same issue of poor generalization on unseen data.

H. Light GBM with Bayesian Optimization:

LightGBM is a gradient boosting framework that is highly efficient and designed for speed and performance. Preprocessing was performed using TF-IDF Vectorizer with max_features=2000 and n_grams = (1,3). To deal with class imbalance, the SMOTE method with sampling_strategy = "not minority" was used to oversample the minority classes to match the majority classes. Since Light GBM requires a lot of parameter tuning, therefore we used "Bayesian Optimization" to find the best parameters i.e. "max_depth", "learning_rate", "n_estimators", "reg_lambda", and "reg_alpha". It is important to note that there are other methods such as Grid Search for hyperparameter tuning but Bayesian Optimization offers a faster and more efficient solution than Grid Search, especially in large or complex hyperparameter spaces. With optimized parameters, the training error was "0.0895" and the validation error was "0.5388", indicating the same issue of poor generalization on unseen data.

I. Decision trees:

The decision tree classifier was implemented using *sklearn*. It was trained using the Gini impurity criterion, with hyperparameter tuning applied to optimize tree depth, to potentially reduce overfitting.

3.4 What we still plan to do?

All the above methods tried so far have provided an initial baseline, but the current misclassification rate is still high. We would like to continue with these methods and use techniques like cross-validation to tune the hyperparameters such as learning rate, batch sizes, and regularization strengths to find the optimal configuration for improving classification accuracy.

Next, we plan to enhance our preprocessing by incorporating word embeddings such as GloVe and FastText. These embeddings will allow us to capture semantic relationships between words, improving the model's ability to understand contextual meaning beyond simple tokenization. Unlike frequency-based methods, these embeddings represent words in continuous vector space.

In case our accuracy does not improve after these changes, we will consider refining the classification task by combining similar emotion categories. Some emotions in the dataset such as annoyed and angry, have overlapping meanings. By manually merging these categories, we can reduce misclassification and improve the performance. This will be a last resort method in case hyperparameter tuning and other preprocessing techniques do not improve performance.

Additionally, we will experiment with deep learning models such as a BERT-based classifier.. BERT is particularly suited for this task as it understands words based on their context in a sentence, making it more effective for conversations.

4. Task Breakdown

In the formal proposal, the project task breakdown included both classification and rewriting empathetic responses. Now, the focus has solely shifted to classification, removing the rewriting task. Baseline classification has been expanded to include hyperparameter tuning, error analysis with a confusion matrix, and testing different preprocessing and feature vectorization methods. The BERT-based classification task remains unchanged.

Task	**Team Member(s)**
Preprocessing the data	Nitin Gupta
Baseline Classification - Hyperparameter tuning - Error & Confusion matrix reporting - Implementation with different preprocessing and feature vectorization	Akina Kuperus, Riya Shrestha
BERT-based Classification	Taimoor Shahzad

5. Initial Results

Table 1 summarizes our initial results for emotion classification.

Model	**Feature Vectorization**	**Training error**	**Validation Error**	
Logistic regression	Word2Vec	0.6747	0.7066	Akina
Linear SVM	Word2Vec	0.7986	0.8125	Akina
Logistic regression	Doc2Vec	0.6744	0.7175	Akina
Linear SVM	Doc2Vec	0.8251	0.8566	Akina
Logistic regression	TF-IDF	0.3222	0.5548	Nitin
Logistic Regression - Balanced Class	TF-IDF	0.003	0.4683	Taimoor
SVM-RBF & SMOTE	TF-IDF	0.0102	0.4987	Taimoor
Random Forest - Balanced Class	TF-IDF	0.0074	0.5458	Taimoor
Light GBM - Bayesian Optimization	TF-IDF	0.0895	0.5388	Taimoor
Random Forest	TF-IDF	0.3868	0.6609	Riya
Naive Bayes	TF-IDF	0.1125	0.5802	Riya
Decision Tree - Gini	TF-IDF	0.0	0.7505	Yakubu

Table 1: A summary of training and validation error using different machine learning models and feature vectorization methods.

These results do not achieve low validation error. To improve the performance we will:

1. Research and test different preprocessing methods.
2. Use cross-validation to tune hyperparameters for each model.
3. Try BERT-based classification.

References

Le, Quoc, and Tomas Mikolov. "Distributed representations of sentences and documents." International conference on machine learning. PMLR, 2014.

Rashkin, Hannah, et al. “Towards Empathetic Open-domain Conversation Models: A New Benchmark and Dataset.” *Association for Computational Linguistics*, vol. Proceedings of the 57th Annual Meeting, 2019, pp. 5370–5381. Accessed 12 February 2025.

Řehůřek, Radim, and Petr Sojka. "Software framework for topic modelling with large corpora." (2010).

Rong, Xin. "word2vec parameter learning explained." arXiv preprint arXiv:1411.2738 (2014).